

MINOR PROJECT REPORT

TopGrade Internship

Student Query Chatbot

A Rule-Based Chatbot for Answering Common Student Queries

Submitted by:

Gopichand Sabbavarapu

Course: Artificial Intelligence

Semester: 7th Semester

Domain: Artificial Intelligence

July 2026

1 Problem Statement

Every semester, students ask the same handful of questions over and over again — what courses are offered, when classes run, how much the fees are, where to reach the admissions office, and so on. Right now, these questions get answered manually, either by visiting the college office, calling up the staff, or waiting for a reply on email. This works, but it isn't always convenient, especially outside office hours or during busy admission periods when the same queries pile up from dozens of students at once.

The problem this project tries to solve is straightforward: build a small program that can instantly answer these repetitive, well-defined questions on its own, without needing a staff member to be available every time.

2 Objective

The objective of this project is to design and implement a simple chatbot that can:

- Accept a student's question as free-text input.
- Understand the intent behind the question using basic keyword-based pattern matching (a lightweight form of NLP).
- Return the correct, pre-written answer for that topic.
- Handle questions it doesn't recognize gracefully, instead of crashing or giving a non-sensical reply.
- Let the student end the conversation whenever they want.
- Work through two different interfaces — a console version for quick testing, and a simple web page version for a more familiar, browser-based experience.

The scope is deliberately kept small. This isn't meant to replace a real support desk — it's meant to demonstrate, at a beginner level, how rule-based NLP and pattern matching can automate a genuinely useful, narrow task.

3 Tools & Technologies Used

Tool / Technology	Purpose
Python 3	Core programming language used for the chatbot logic
CSV (comma-separated values)	Storage format for the prepared question–answer dataset
Python <code>string</code> module	Cleaning user input (removing punctuation, lowercasing)
Python <code>csv</code> module	Reading the dataset file at runtime
Flask	Lightweight Python web framework used to build the optional web interface
HTML, CSS, JavaScript	Front-end of the web chat interface
VS Code	Code editor used for writing and testing the project
Git & GitHub	Version control and hosting the project repository

4 System Flow / Algorithm

The chatbot follows a simple, linear flow every time it receives a message from the user. The steps below describe exactly what happens from the moment the program starts to the moment it ends.

Algorithm

1. **Start** the program and load the prepared question–answer dataset from the CSV file into memory.
2. **Read** the student’s question as raw text input (from the console, or from the web page’s input box).
3. **Clean** the input — convert it to lowercase and strip out punctuation, so that “What are the Fees?” and “what are the fees” are treated identically.
4. **Compare** the cleaned input against the list of keyword groups stored in the dataset (for example, the keywords *fee*, *fees*, *cost*, *payment* are all grouped under one topic).
5. **Decide:**
 - If a keyword from any group is found in the input, return that group’s stored answer.
 - If no keyword matches anything, return a default fallback message asking the student to rephrase or pick a listed topic.
6. **Display** the response to the student.
7. **Check** if the input was an exit command (*bye*, *exit*, *quit*). If yes, print a goodbye message and stop. If no, go back to Step 2 and wait for the next question.

This is a classic rule-based approach rather than a machine-learning one — there is no training involved. The “intelligence” here comes entirely from how the dataset’s keyword groups are designed, which is why preparing a good, well-thought-out dataset (Step 1) matters just as much as the matching logic itself.

5 Implementation Steps

1. **Identify common student questions.** Before writing any code, a list of the questions students realistically ask was drawn up — covering courses, timings, fees, contact details, admissions, hostel, library, placements, exams, and scholarships.
2. **Prepare the dataset.** Each topic was mapped to a set of keywords that students might use to ask about it in different ways, along with one accurate answer. This was stored in a structured `qa_dataset.csv` file, with one row per topic.
3. **Build the input-cleaning function.** A small function was written to lowercase the input and remove punctuation, since students rarely phrase questions with consistent capitalization or punctuation.
4. **Build the matching function.** This function loops through every topic’s keyword list and checks whether any keyword appears in the cleaned input. The first match found determines the response.
5. **Build the main console loop.** Using Python’s built-in `input()` function, the chatbot repeatedly asks for a question, matches it, prints the answer, and checks for an exit command, looping until the student decides to stop.
6. **Add a fallback response.** If nothing matches, the chatbot doesn’t fail silently — it tells the student it didn’t understand and lists the topics it can help with.
7. **Build the web interface (optional extension).** Using Flask, the same matching logic was reused (imported, not rewritten) and wrapped behind a simple web page with a text box and a chat window, so the same chatbot can be used through a browser instead of a terminal.
8. **Test thoroughly.** Each topic was tested with multiple phrasings to confirm the keyword groups were broad enough to catch natural variations, and the exit command was tested to make sure the loop terminates cleanly.

6 Output Screenshots

Figure 1: Chat window showing a full conversation

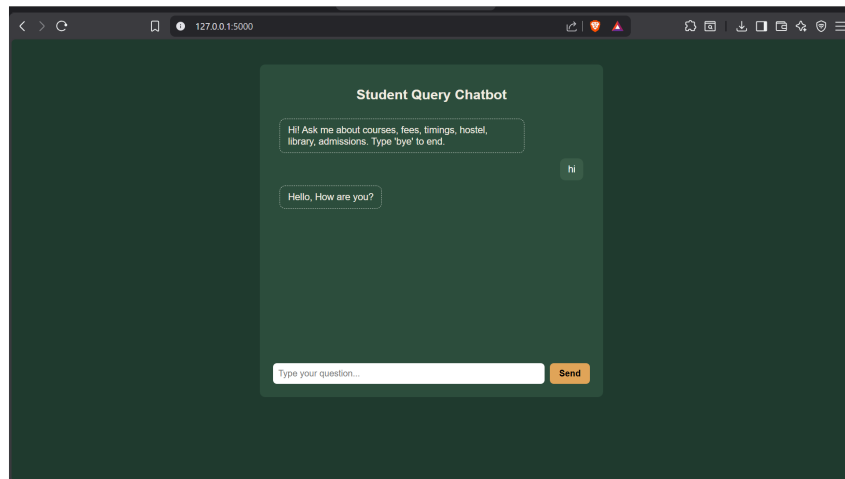


Figure 2: User input being typed and sent

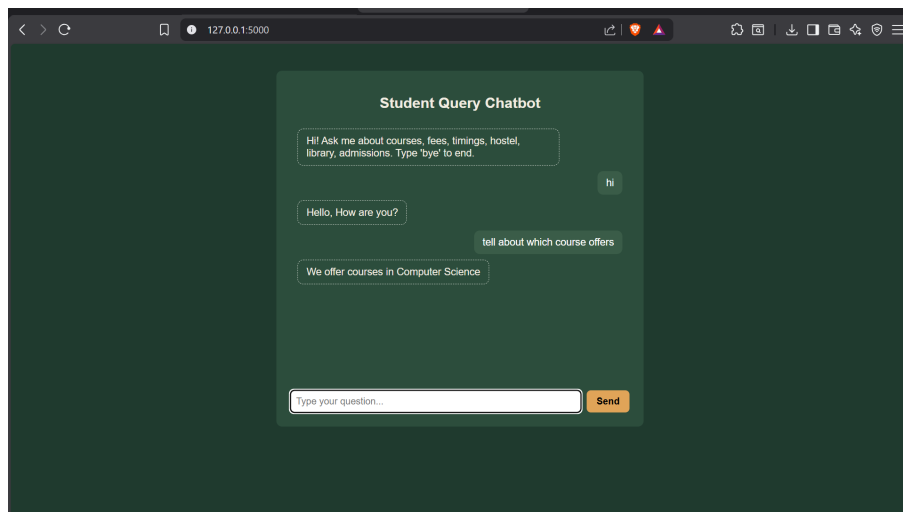
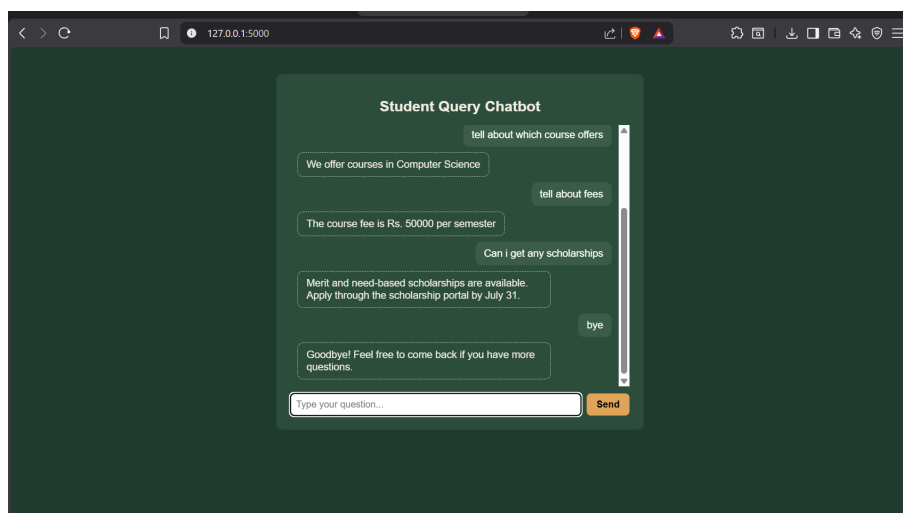


Figure 3: Bot response displayed to the user



7 Conclusion

This project set out to build a simple, working chatbot that could take over the repetitive job of answering common student questions, and it does exactly that. Using nothing more than keyword matching and a well-organized dataset, the chatbot correctly identifies the topic behind a student's question and responds with accurate information, whether it's asked through the console or through a small web page.

The project also reinforced a few practical lessons — that a rule-based system is only as good as the dataset behind it, that cleaning user input (handling case and punctuation) matters more than it might seem at first, and that keeping the data separate from the logic (in a CSV file rather than hardcoded into the program) makes the whole system much easier to extend later.

There's plenty of room to take this further: adding real NLP techniques like stemming or similarity scoring instead of exact keyword matches, hooking it up to a live database instead of a static CSV, or expanding the topic list to cover more of what students actually ask. But even in its current, deliberately simple form, the chatbot fulfills its core purpose — giving students fast, consistent answers to the questions they ask most.